PLDI Seoul 2025

UST ETRI Electronics and Telecommunications Research Institute

Multi-level Machine Learning-Guided Autotuning for Efficient Code Generation on a Deep Learning Accelerator

JooHyoung Cha

UST

JooHyoung Cha, Munyoung Lee, Jinese Kwon, Jemin Lee, Yongin Kwon*
University of Science and Technology, Electronics Telecommunications Research Institute

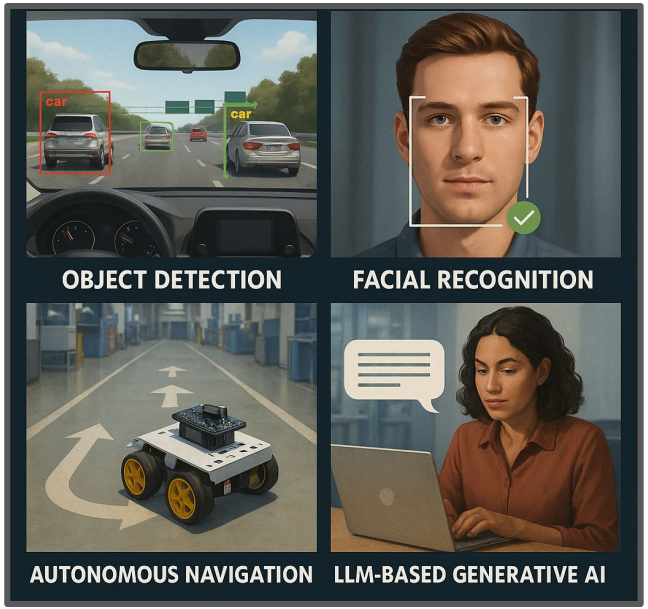
Outline

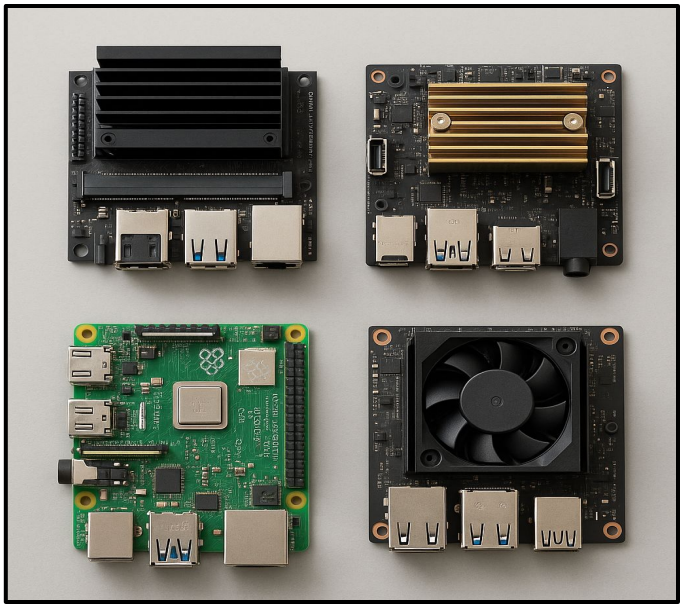
1. Motivation
2. System Architecture Overview
3. Methodology
4. Evaluation
5. Summary

Background

UST ETRI Electronics and Telecommunications Research Institute

Deep Learning Inference on Embedded Systems





The systems must be optimized across accuracy, speed, and cost!!

Background

What is Autotuning?

Algorithm	Code (Schedule)	Hardware
What is computed	How it is computed	Where it is computed

$$C_{ij} = \sum_k A_{ki} B_{kj}$$

$$\text{softmax}_i = \frac{\exp(X_i - \max_j X_j)}{\sum_k \exp(X_k - \max_j X_j)}$$

```
for y in range(128):
  for x in range(128):
    # Tiling Strategy
    # double buffering
    # etc …
```

Automatic generation of a search space of possible schedules to identify the most optimal.

Background

Architecture of VTA[1)]

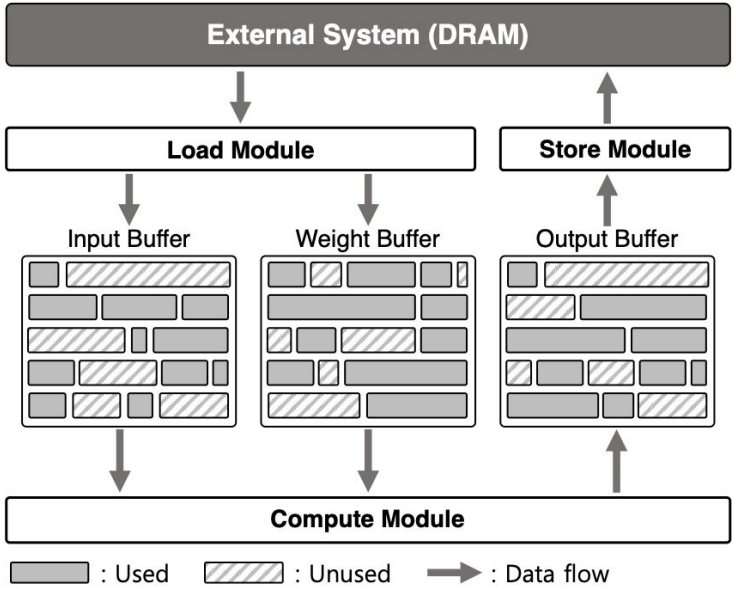
(1) **Load/Store** Module

1. Transfers data between **DRAM** and **Buffers**

(2) **Compute** Module

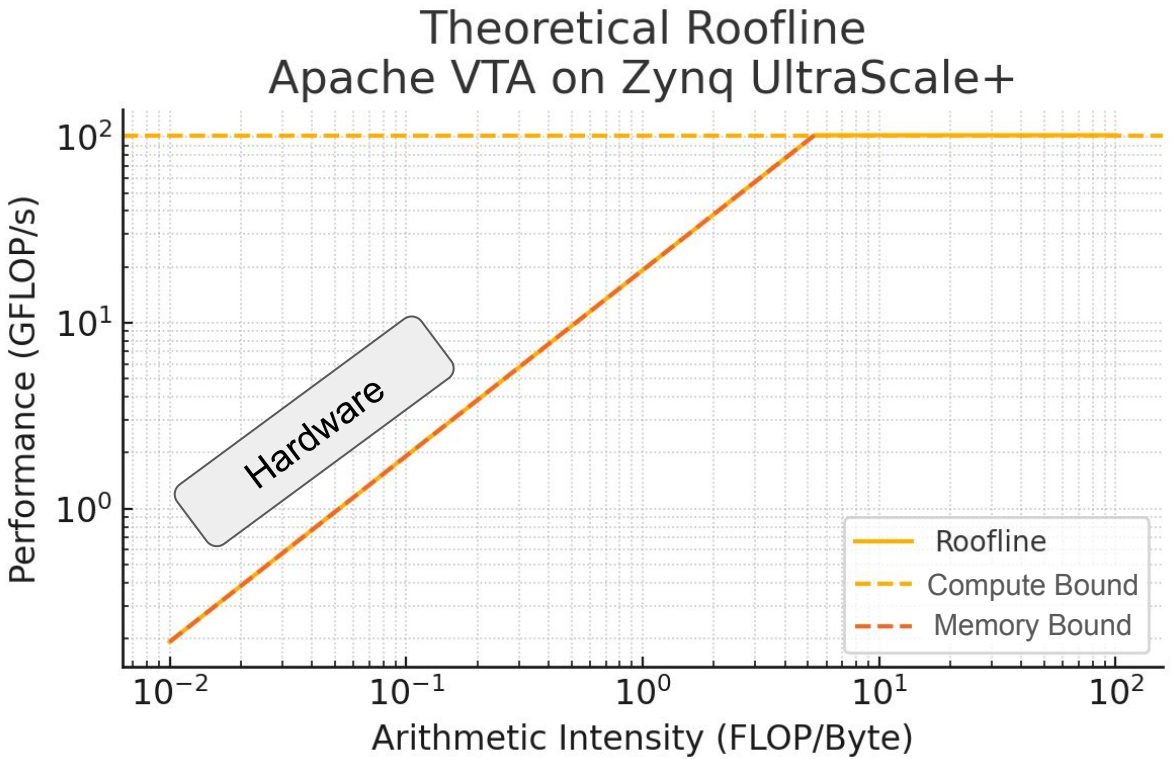
1. Performs matrix multiplication on data from **Buffers**
2. Accumulates the results to **Output Buffer**

All modules operate in parallel!!

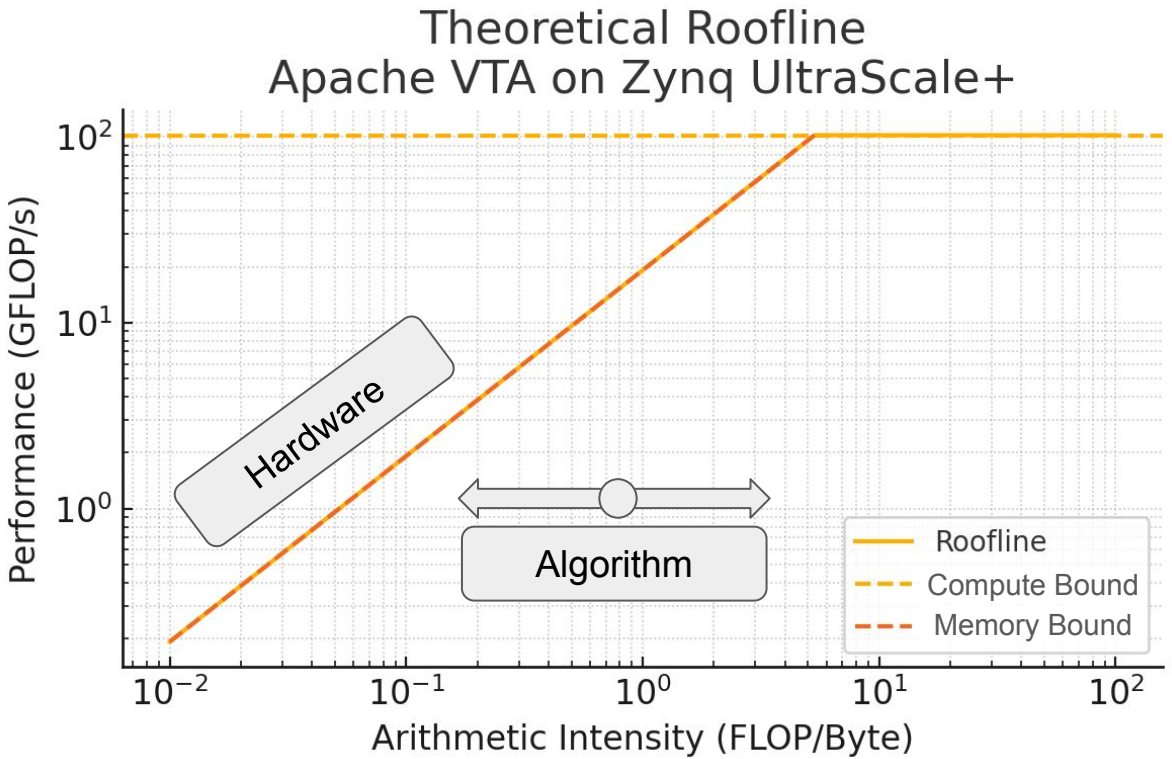


1) Thierry Moreau et al., 2019. A Hardware-Software Blueprint for Flexible Deep Learning Specialization.

Background

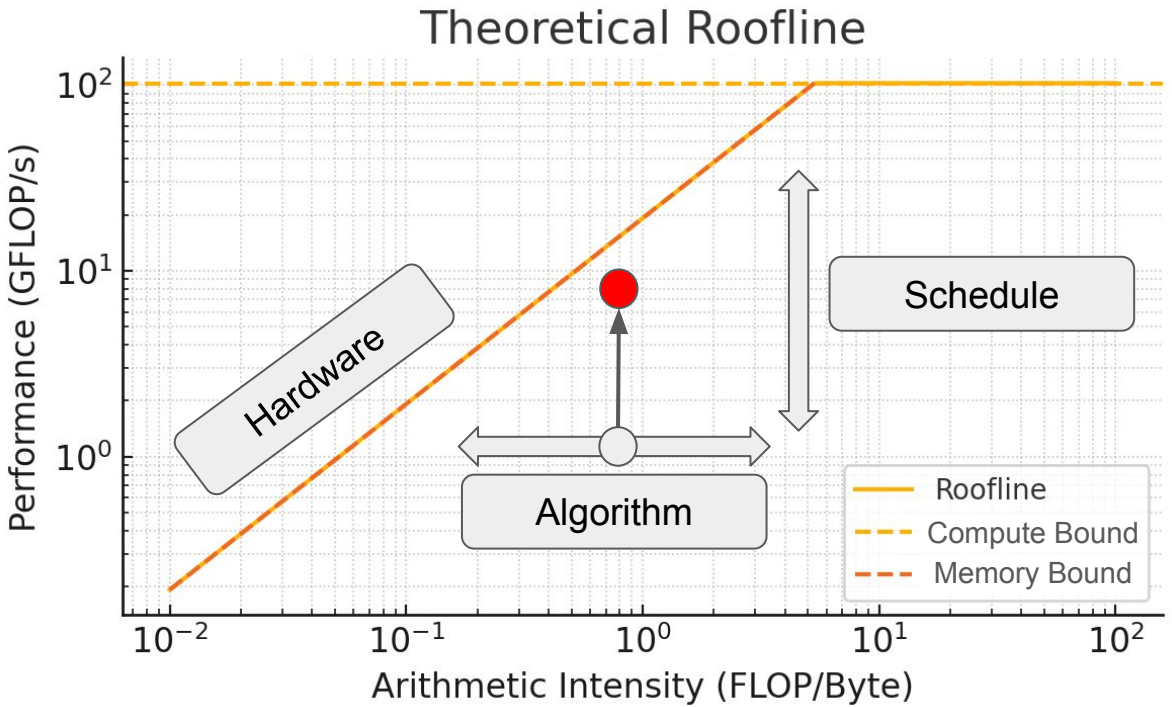

Theoretical Roofline
Apache VTA on Zynq UltraScale+
Performance (GFLOP/s)
10^2
10^1
10^0
Hardware
Roofline
Compute Bound
Memory Bound
10^{-2}
10^{-1}
10^0
10^1
10^2
Arithmetic Intensity (FLOP/Byte)

Background


Theoretical Roofline
Apache VTA on Zynq UltraScale+
Performance (GFLOP/s)
10^2
10^1
10^0
Hardware
Algorithm
Roofline
Compute Bound
Memory Bound
10^{-2}
10^{-1}
10^0
10^1
10^2
Arithmetic Intensity (FLOP/Byte)

Motivation

Schedule tuning is the KEY to improve performance

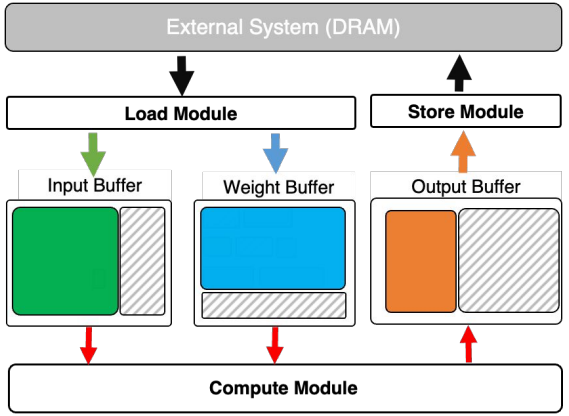


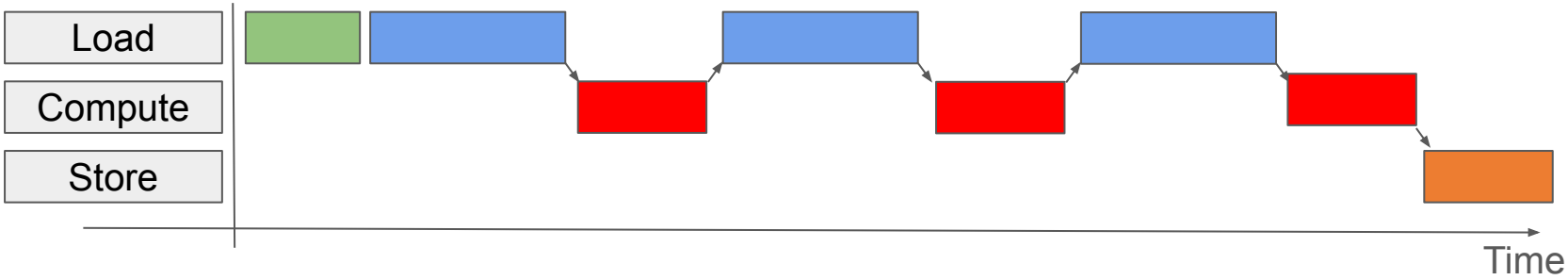
Motivation

UST ETRI
Electronics and Telecommunications Research Institute

A (Virtual) Thread

1. Allocate **Input** / **Weight** / **Output** (on-chip Buffers)
2. Load **Input** and **Weight** from DRAM to the Buffers
3. **Compute** Matrix-Multiplication
4. Store **Output** to DRAM and Reuse Buffers

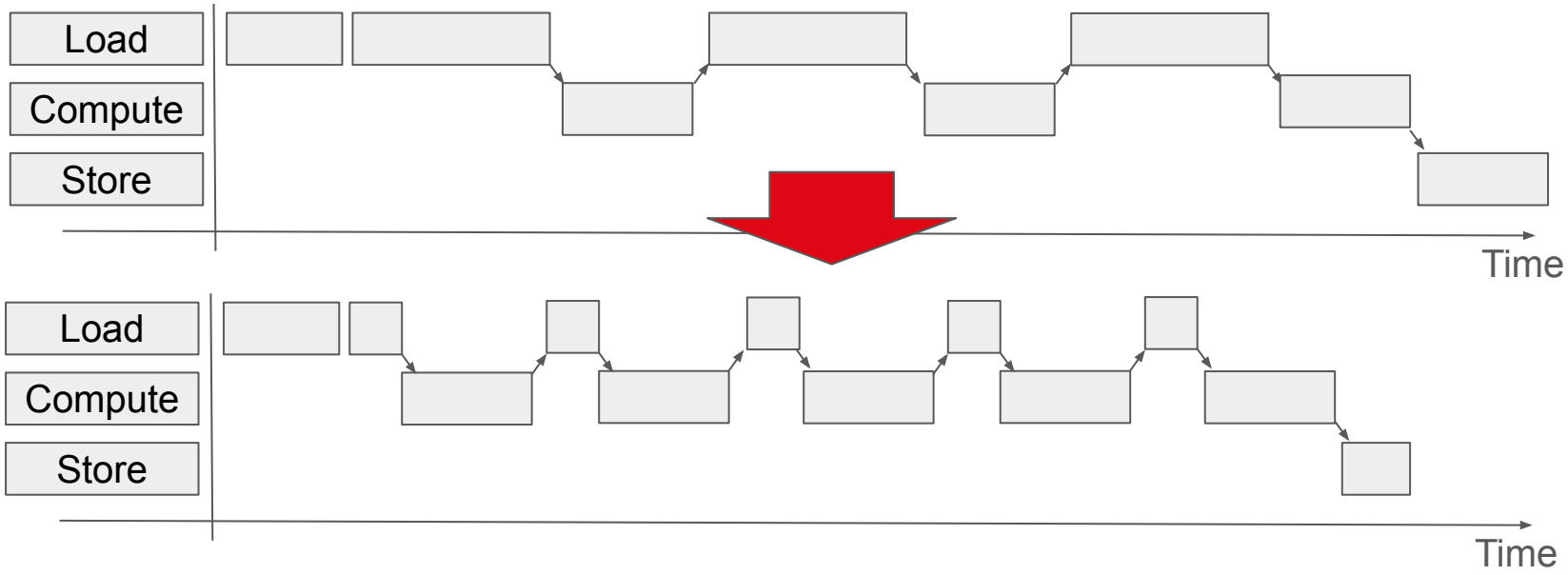




Motivation

Tuning Knob #1: Tiling

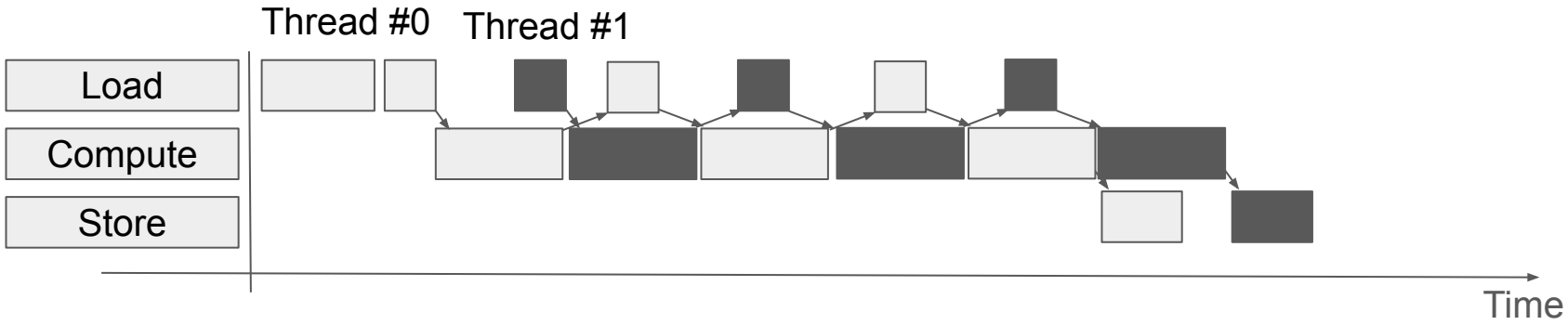
Optimizes execution by dividing tasks into smaller tiles



Motivation

Tuning Knob #2: Virtual Threading

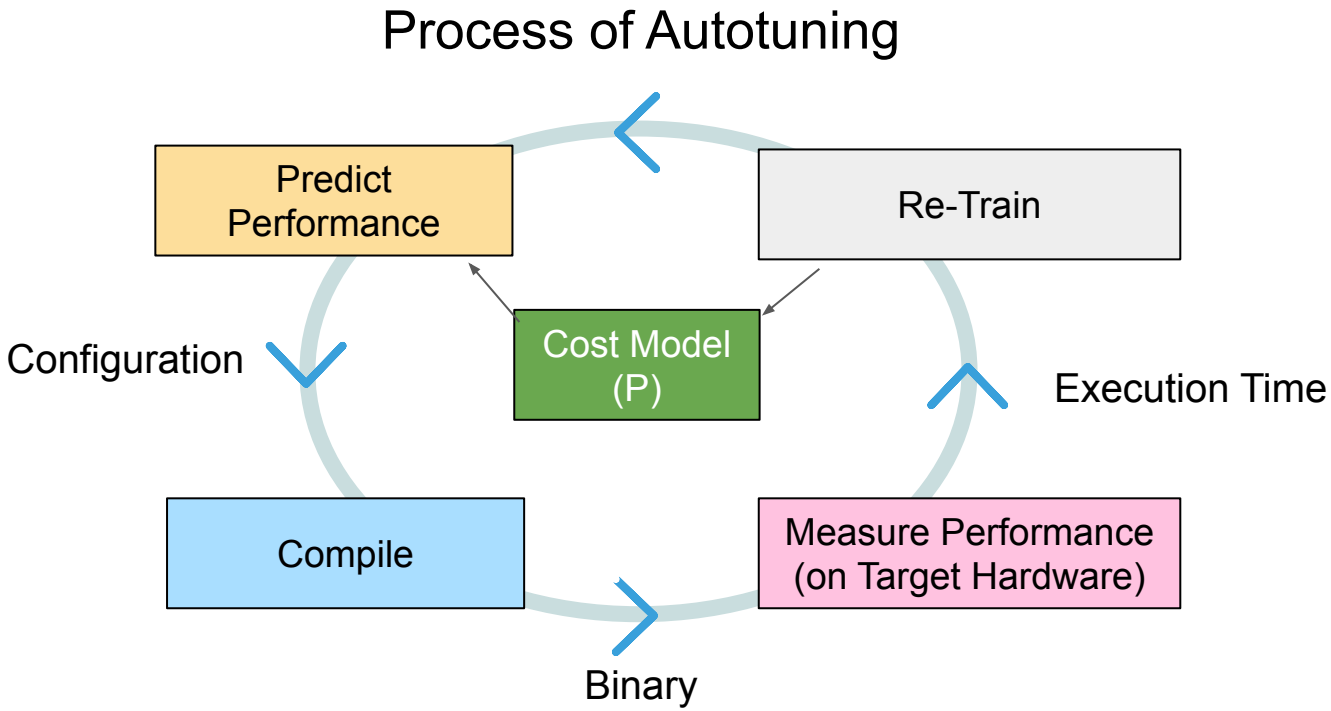
1. Virtual Threading works to keep each Module running without pause.
2. Overlapping load/store of other threads hides memory latency.



Each execution **Configuration** is formed by a combination of **Tuning Knob values**

Motivation

Process of Existing Autotuning



Challenges

Why Tuning Is Challenging

1. **Explosive Search Space**
   Countless knob combinations generate diverse schedules and memory layouts.

2. **High Invalidity Rate**
   Hardware limits and schedule conflicts can lead to buffer-allocation failures and timing errors.

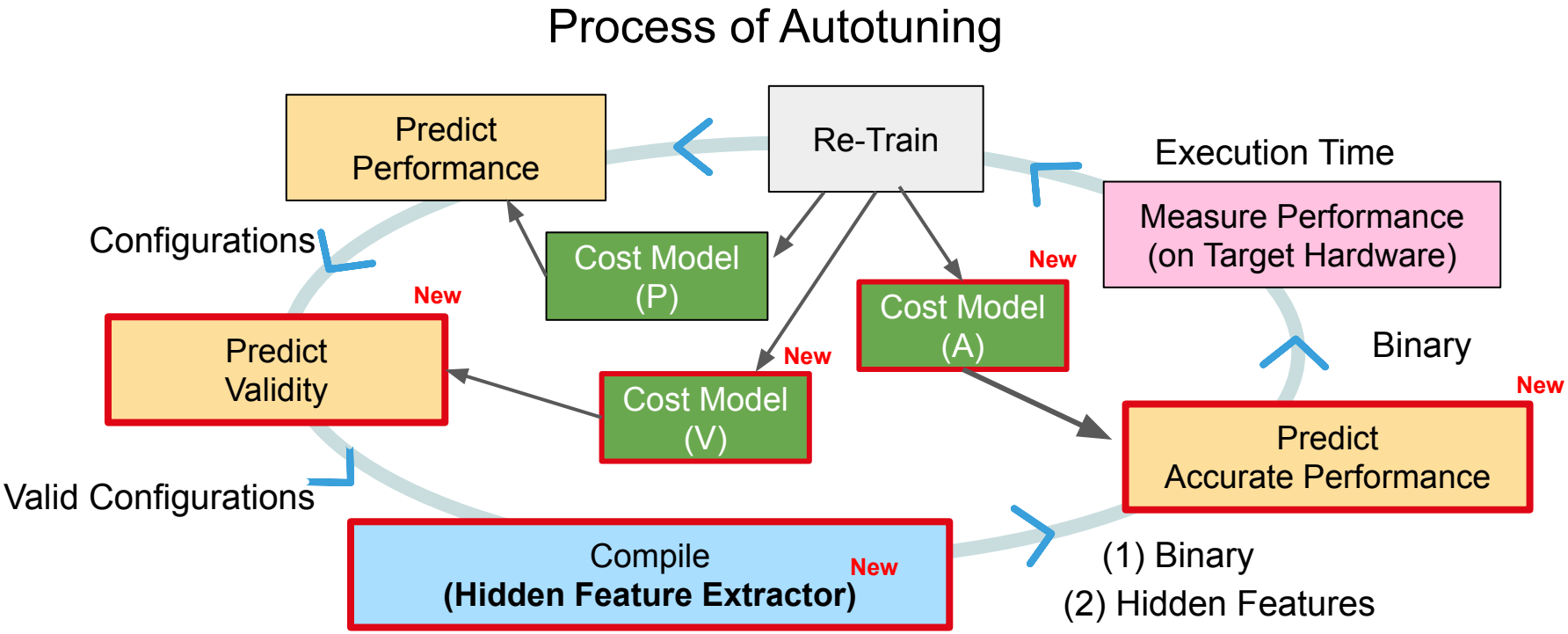
3. **Critical Failure Mode**
   An invalid configuration can crash the hardware and force a full reboot.

FAILURE
make delays the overall tuning workflow.

Model	Name	Input (H, W, C)	Kernel (N, H, W)	Output (H,W)	Padding, Stride	Invalidity Ratio
ResNet18	Conv 1	56, 56, 64	64, 3, 3	56, 56	1, 1	82.64%
	Conv 2	56, 56, 64	128, 1, 1	28, 28	0, 2	79.66%
	Conv 3	56, 56, 64	128, 3, 3	28, 28	1, 2	80.57%
	Conv 4	28, 28, 128	128, 3, 3	28, 28	1, 1	69.35%
	Conv 5	28, 28, 128	256, 3, 3	14, 14	1, 2	52.49%
	Conv 6	14, 14, 256	512, 3, 3	7, 7	1, 2	50.47%
	Conv 7	7, 7, 512	512, 3, 3	7, 7	1, 1	50.47%
InceptionV3	Conv 1	73, 73, 64	80, 1, 1	73, 73	0, 1	83.68%
	Conv 2	149, 149, 32	32, 3, 3	147, 147	0, 1	80.37%
	Conv 3	35, 35, 48	64, 5, 5	35, 35	2, 1	77.48%
	Conv 4	8, 8, 384	384, 3, 1	8, 8	1, 1	32.81%
	Conv 5	17, 17, 192	192, 1, 7	17, 17	0, 1	51.72%
	Conv 6	17, 17, 192	192, 7, 1	17, 17	3, 1	53.06%

Methodology

Overview of Proposed Autotuning



Methodology

Challenges

(1) A high ratio of **Invalid Configurations** increases tuning time.

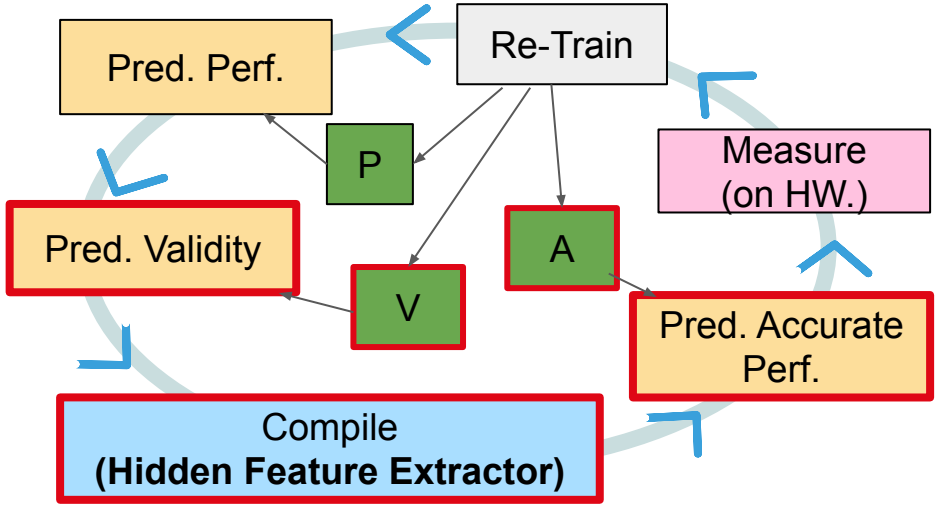
(2) **High-level Features** are **difficult** to predict performance for Autotuning

Proposed Method

(1) Reduce Invalid Conf. by Validity Model

(2.1) Extract Hidden Features by CodeGen

(2.2) Hidden Features used to Advanced Model



Methodology

Challenges

(1) A high ratio of **Invalid Configurations** increases tuning time.

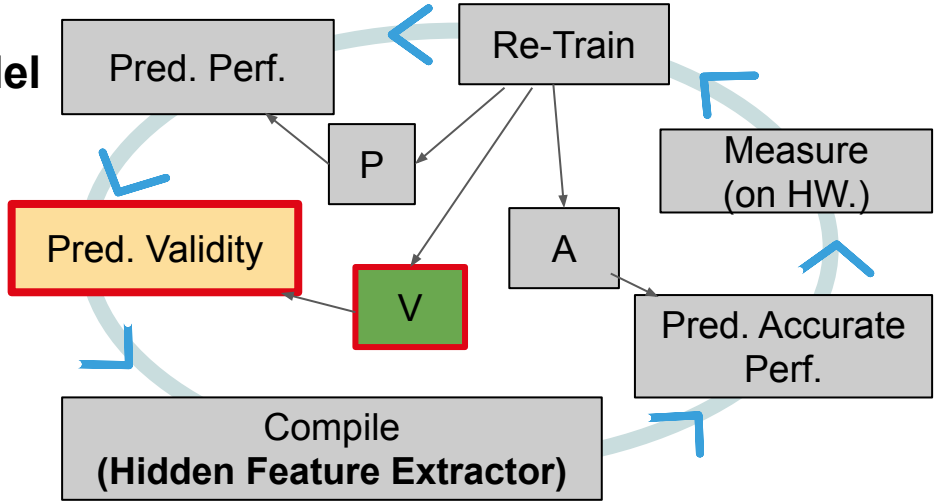
(2) **High-level Features** are **difficult** to predict performance for Autotuning

Proposed Method

(1) Reduce Invalid Conf. by Validity Model

(2.1) Extract Hidden Features by CodeGen

(2.2) Hidden Features used to Advanced Model



Methodology

Challenges

(1) A high ratio of **Invalid Configurations** increases tuning time.

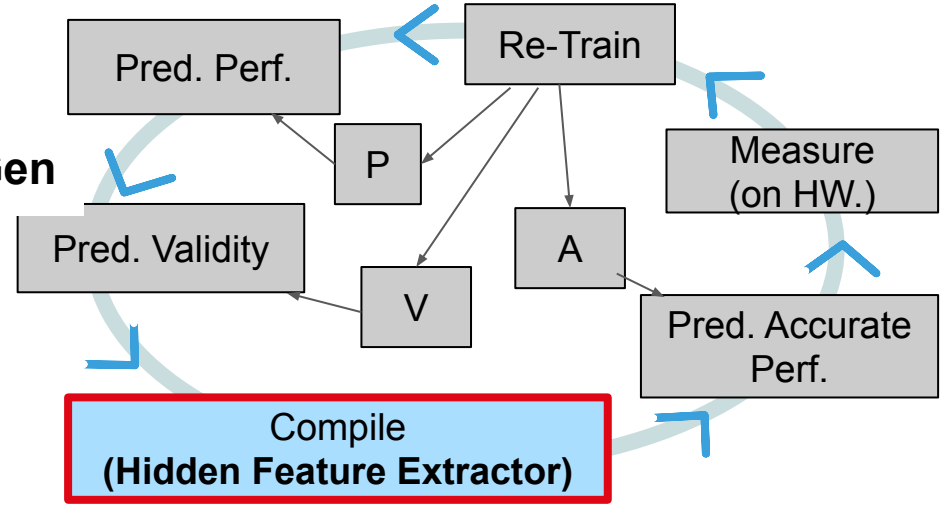
(2) **High-level Features** are **difficult** to predict performance for Autotuning

Proposed Method

(1) Reduce Invalid Conf. by Validity Model

(2.1) Extract Hidden Features by CodeGen

(2.2) Hidden Features used to Advanced Model



Methodology

Challenges

(1) A high ratio of **Invalid Configurations** increases tuning time.

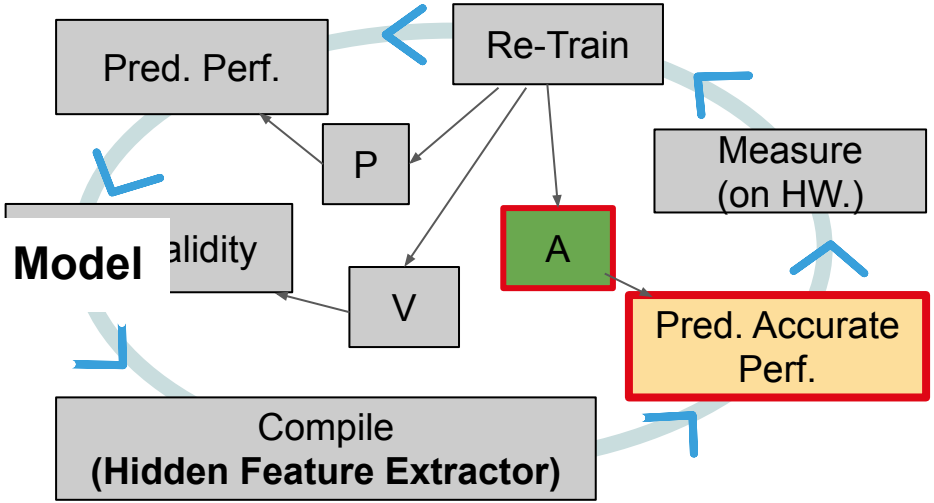
(2) **High-level Features** are **difficult** to predict performance for Autotuning

Proposed Method

(1) Reduce Invalid Conf. by Validity Model

(2.1) Extract Hidden Features by CodeGen

(2.2) Hidden Features used to Advanced Model



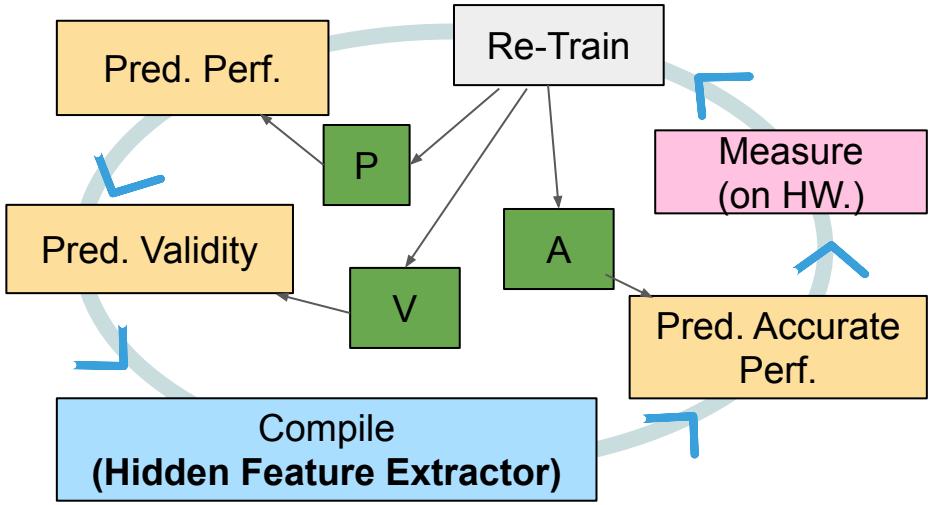
Experimental Setup

Hardware Platform : VTA on XILINX ZCU 102

Deep Learning Compiler : NEST-C (based on GLOW)

Cost Model : DMLC XGBoost (used to Model (P, V, A))

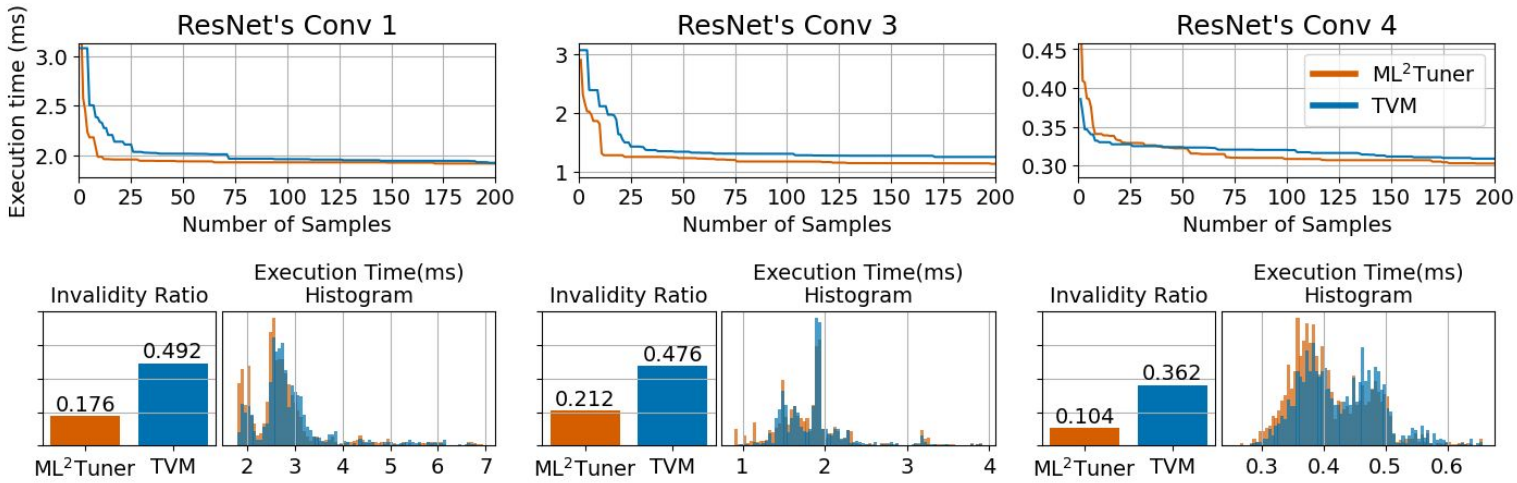
Experimental Model : ResNet18, InceptionV3



Experiments

Evaluation - ResNet18

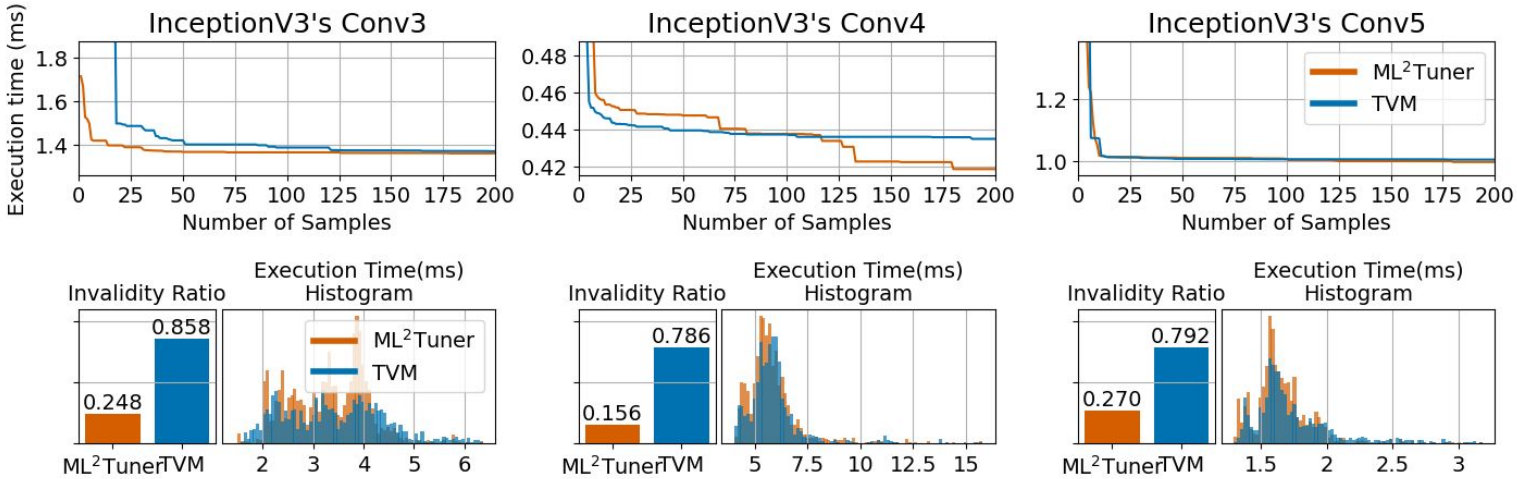
- Invalid Configurations are filtered by Model V at Compile time.
- Exploring more opportunities for Exploration and better Configuration.
- Average of Invalidity Ratio 60.8 % ↓



Experiments

Evaluation - InceptionV3

- The optimization performance is consistent across other models.
- From left to right, the kernel filters have shapes 5×5, 1×3, and 1×7.

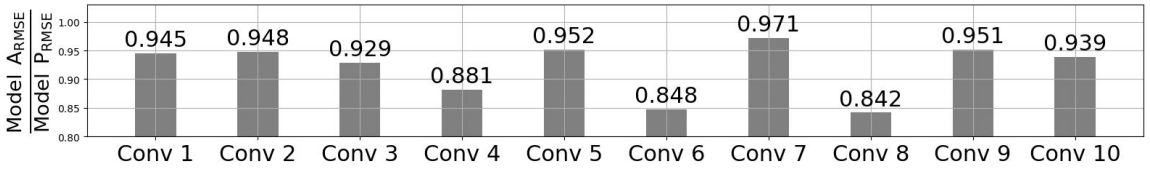


Experiments

Evaluation - Hidden Feature

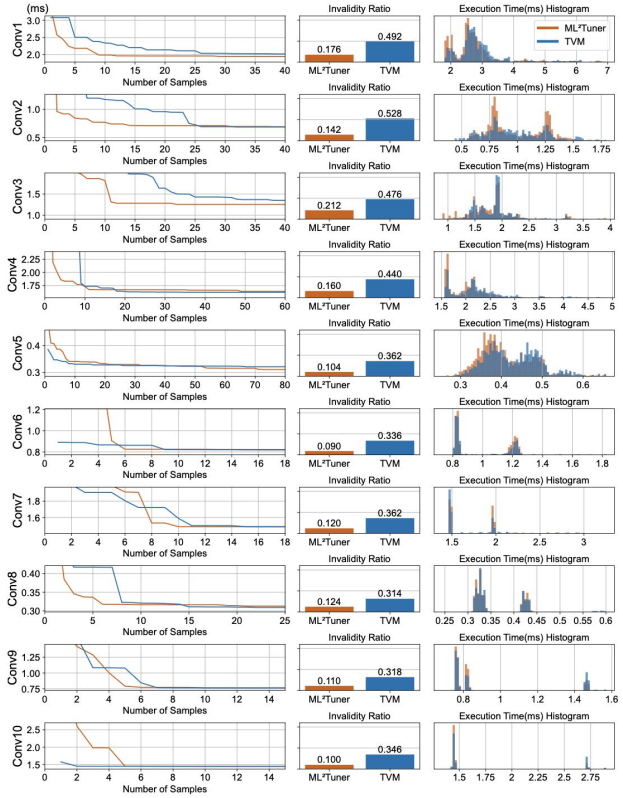
- Hidden features improve the performance prediction accuracy.
- RMSE(Loss) was computed in XGBoost using the same number of samples.

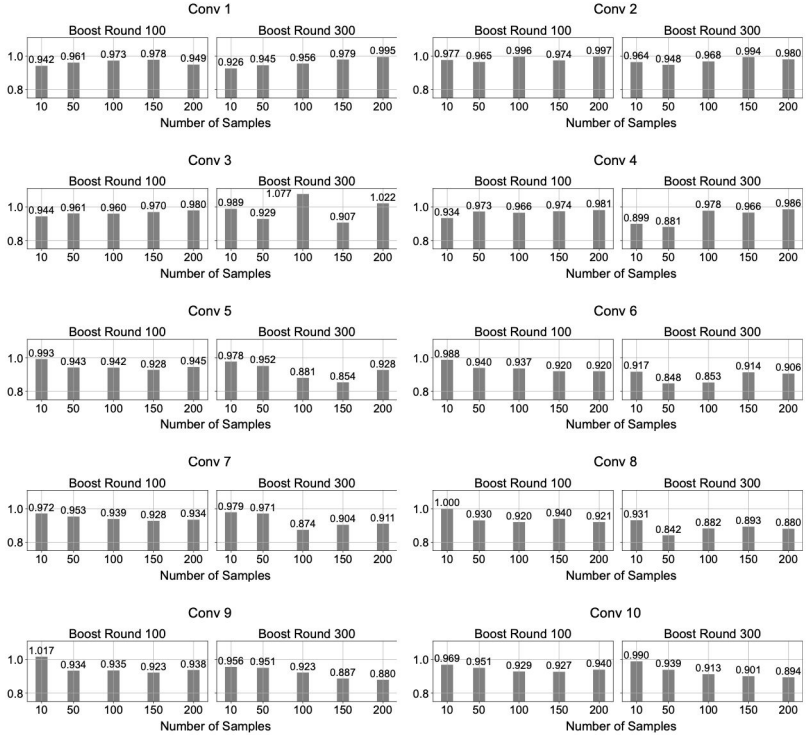
Performance Comparison between Model P and Model A (in ResNet18)



Set Boost Round of XGBoost to 300

Experiments





Summary

- **Multi-level ML and Hidden Feature**
  The proposal method reduces invalid configurations and tuning time.

- **Reducing Invalidity Ratio**
  Model Validity reduce the rate of Invalid configurations by 60.8 %

- **Hidden Feature extracted from Compiler**
  Compiler produces hidden features Automatically.
  Model A with Hidden Feature reduce loss by 8.4 %

PLDI Seoul 2025

UST ETRI Electronics and Telecommunications Research Institute

THANK YOU

Backup

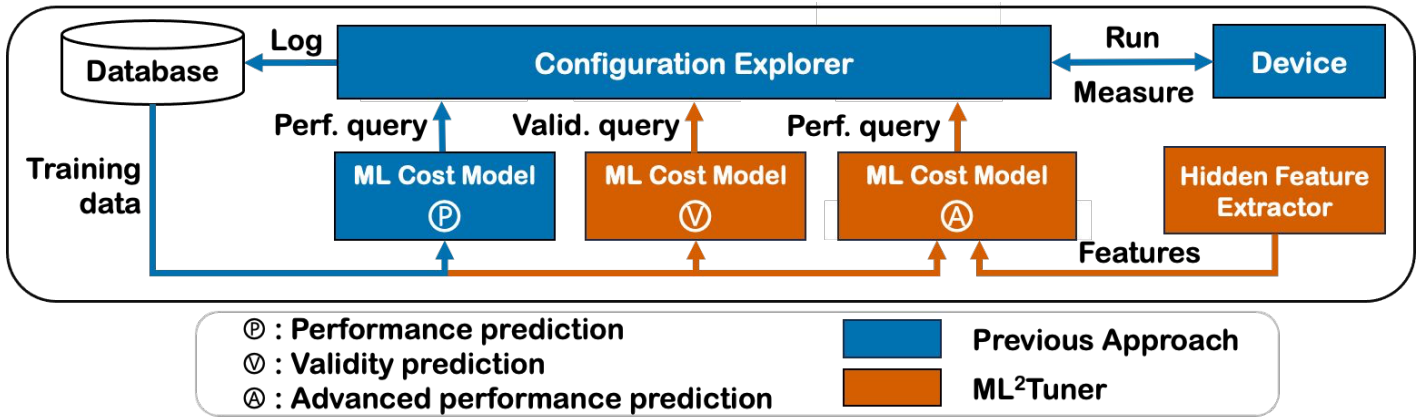
ML2Tuner consists of several components:

(1) **Model P** : Feature-Based Optimal Configuration Search

(2) **Model V** : Feature-Based Configuration Validation

(3) **CodeGen** : Program Generation with Hidden Feature Extraction

(4) **Model A** : Feature & Hidden Feature-Based Optimal Configuration Search



Backup

Hidden Feature Extractor

(1) Variables created and derived from the flow of the code

int **value** = 30; → Capture the variable **value** as a hidden feature.

(2) Variables that affect a specific control flow branch (IF)

if (**value**) { … } → Capture the **condition** variable **value** as a hidden feature.

(3) Variables instantiated only along a specific control flow branch (in IF)

if (value) { int **value** = 40; } → Capture the {variable **value** in **IF**} as a hidden feature.

(4) Variables from Loop counts (For - Loop Counts)

for (int i = 0 ; i < **value**; i++) { } → Capture the **value** as a hidden features.

Backup

Hidden Features

(bold : tuning knobs, non-bold : hidden features)

Feature	GeoAVG	Normalized Feature Importance Score (%)									
		Conv1	Conv2	Conv3	Conv4	Conv5	Conv6	Conv7	Conv8	Conv9	Conv10
Tile width	29.268	19.685	27.412	23.810	25.779	32.631	30.633	28.432	31.596	31.975	32.582
Tile height	25.925	15.256	19.768	20.186	22.288	27.235	29.335	29.201	32.406	31.028	30.622
Loop count (Thread creation)	8.468	10.581	8.434	9.058	9.130	6.937	6.231	7.941	8.101	7.579	7.594
Parallelism factor	8.194	5.413	7.907	7.505	7.519	7.965	7.269	7.941	8.912	9.237	8.574
Variable Value (Derived from Num. of kernels and threads)	4.933	7.382	5.271	5.435	4.565	3.854	4.154	3.842	4.321	4.737	3.920
Variable Value (Drived from output size variables)	4.083	2.215	3.163	3.364	3.491	4.625	4.673	4.611	4.861	4.737	4.655
Variable Value (Drived from output size variables)	4.166	2.707	4.217	3.364	3.491	5.139	4.413	4.098	4.051	4.500	4.655
Variable Value (Partial input tile width)	3.069	13.287	6.326	7.764	6.176	1.542	2.336	3.330	0.810	0.711	0.735
Variable Value (Partial output tile height in case A)	1.946	0.738	1.581	1.812	2.685	2.569	2.596	2.561	1.080	1.184	1.715
Loop count (Loop for output store instructions)	1.946	1.722	2.636	2.588	2.954	1.542	2.336	2.305	0.810	0.711	0.980
Variable Value (Resized output tile height in case A)	1.233	9.104	3.426	3.623	1.611	0.257	0.260	0.512	0.270	0.237	0.245
Variable Value (Partial output tile height in case B)	1.069	1.476	1.845	3.623	2.148	0.771	0.779	1.025	0.270	0.237	0.245
Variable Value (Drived from Num. of kernels and threads)	0.740	0.492	0.527	0.518	0.806	1.799	1.038	0.768	0.270	0.474	0.490
Variable Value (Drivced form tile width)	0.411	0.246	0.264	0.259	0.269	0.514	0.519	0.256	0.270	0.474	0.490
Variable Value (Resized output tile height in case B)	0.274	0.738	0.791	0.776	1.343	0.257	0.260	0.256	0.027	0.024	0.024
Variable Value (Drivced form tile height)	0.384	0.246	0.264	0.259	0.269	0.514	0.519	0.256	0.270	0.474	0.490
Variable Value (Resized input tile height in case A)	0.274	1.230	0.527	0.518	0.215	0.026	0.026	0.077	0.054	0.024	0.024
Variable Value (Resized input tile height in case B)	0.055	0.098	0.105	0.104	0.161	0.026	0.026	0.026	0.001	0.000	0.000